Especificação do Protocolo BCP (Blockchain Copy Protocol)

1. Visão Geral

O **BCP (Blockchain Copy Protocol)** é um protocolo da camada de aplicação concebido para gerenciar a sincronização, distribuição e consenso de uma blockchain simplificada em uma rede Peer-to-Peer (P2P). O protocolo estabelece as regras de comunicação entre nós para a descoberta de pares, solicitação de cópias completas da cadeia de blocos e transmissão proativa de novas atualizações mineradas.

2. Arquitetura e Papéis P2P

Em conformidade com a arquitetura P2P pura, cada nó na rede atua simultaneamente como **Servent** (Servidor + Cliente), desempenhando dois papéis operacionais distintos:

- **Papel de Servidor:**
  - Escuta conexões de entrada em uma porta TCP dedicada (exemplo: porta 8333).
  - Atende a solicitações de novos peers ou de peers já estabelecidos.
  - Responde a requisições de estado, fornecendo a lista de peers conhecidos ou enviando a cópia local da blockchain.
- **Papel de Cliente:**
  - Inicia conexões de saída em direção a IP/Porta de outros peers conhecidos.
  - Executa o processo de inicialização (*bootstrap*) e descoberta de topologia.
  - Solicita a cópia completa da blockchain para sincronização de estado.
  - Realiza *broadcast* de novos blocos minerados para os peers conectados.

3. Camada de Transporte e Formato de Comunicação

O BCP utiliza o protocolo de transporte **TCP** para garantir a entrega confiável, ordenada e sem perdas de mensagens de rede. A troca de dados ocorre por meio do mecanismo nativo de serialização de objetos Java utilizando ObjectInputStream e ObjectOutputStream.

4. Sintaxe das Mensagens

Toda mensagem trafegada no protocolo BCP deve ser encapsulada em uma instância da classe serializável NetworkMessage.

4.1. Estrutura do Pacote (NetworkMessage)

Campo	Tipo	Descrição
type	MessageType (Enum)	Identificador do

CamScanner

Campo	Tipo	Descrição
		comando/operação da mensagem.
payload	Object (Serializable)	Carga útil da mensagem, cujo tipo específico depende do valor de type.

4.2. Tipos de Mensagem (MessageType)

O enum MessageType define as seguintes operações do protocolo:

- HELLO: Mensagem inicial de handshake e apresentação.
- GET_PEERS: Solicitação de lista de endereços de peers conhecidos.
- PEERS_LIST: Resposta contendo lista de peers conhecidos.
- GET_BLOCKCHAIN: Solicitação da cópia completa da blockchain local.
- BLOCKCHAIN_COPY: Envio/Anúncio da cópia serializada da blockchain.
- REJECT: Notificação de rejeição de mensagem ou de cadeia inválida.
- ACK : quando dá certo

5. Semântica das Mensagens

A tabela a seguir especifica o comportamento, o tipo do payload e a finalidade de cada tipo de mensagem no protocolo BCP:

Tipo de Mensagem	Origem	Estrutura do Payload	Semântica e Propósito
HELLO	Cliente	String	Enviada ao abrir a conexão para autenticar a versão do protocolo e fornecer o ID do nó.
GET_PEERS	Cliente	null	Solicita ao nó remoto sua lista atualizada de pares ativos na rede P2P. O código da aplicação precisa ter gravado estaticamente pelo menos 1 endereço semente (Seed Node).
PEERS_LIST	Servidor	List<String>	Retorna uma lista de

CamScanner

Tipo de Mensagem	Origem	Estrutura do Payload	Semântica e Propósito
			strings formatadas no padrão "IP:Porta" com os endereços de outros nós conhecidos.
GET_BLOCKCHAIN	Cliente	null	Solicita a cópia integral do ArrayList<Block> armazenado no nó servidor.
BLOCKCHAIN_COPY	Ambos	ArrayList<Block>	Envia a cópia serializada da cadeia. Utilizada como resposta a GET_BLOCKCHAIN ou em transmissão proativa (*broadcast*) após a mineração de um novo bloco.
REJECT	Servidor	String	Notifica que a cadeia ou mensagem recebida foi rejeitada, contendo o motivo em formato texto (ex.: "Cadeia inválida" ou "Tamanho menor que a cadeia local").

6. Fluxos de Operação do Protocolo

6.1. Fluxo A: Bootstrap e Descoberta de Peers

1. O Nó A inicia e conecta-se ao Nó B via Socket TCP.
2. O Nó A envia NetworkMessage(HELLO, "V1.0-NodeA").
3. O Nó B confirma o handshake.
4. O Nó A envia NetworkMessage(GET_PEERS, null).
5. O Nó B responde com NetworkMessage(PEERS_LIST, ["192.168.1.10:8333", "192.168.1.11:8333"]).
6. O Nó A adiciona os novos endereços à sua lista de descoberta.

CamScanner

6.2. Fluxo B: Sincronização de Estado (Regra da Maior Cadeia Válida)

1. O Nó A envia NetworkMessage(GET_BLOCKCHAIN, null) para o Nó B.
2. O Nó B serializa seu ArrayList<Block> e responde com NetworkMessage(BLOCKCHAIN_COPY, localChain).
3. O Nó A recebe a cadeia transmitida e executa a verificação local:
    - **Validação de Criptografia e Integridade:** Executa isChainValid() na cadeia recebida (valida hashes individualmente e o encadeamento previousHash).
    - **Regra de Consenso:** Se a cadeia recebida for válida **E** seu tamanho for maior que a cadeia local (recebida.size() > local.size()), o Nó A substitui sua cadeia local pela recebida.
    - Se a cadeia for inválida ou menor/igual, o Nó A descarta e envia uma mensagem REJECT.

6.3. Fluxo C: Anúncio de Novo Bloco e Propagação

1. O Nó A conclui o Proof of Work (PoW) de um novo bloco e o insere na sua lista local.
2. O Nó A transmite uma mensagem NetworkMessage(BLOCKCHAIN_COPY, localChain) para todos os peers conectados na sua lista.
3. Cada peer receptor valida a nova cadeia e atualiza seu estado caso a nova cadeia seja válida e superior em comprimento.

Entregáveis e modo de trabalho: Trabalho em dupla; apresentação da aplicação com n peers, sendo n > 4, apresentar em laboratório a estrutura P2P, preferencialmente em vários computadores; usar rede local; caso julgue necessário troque a serialização de objetos por JSON. Apresentação de forma prática usando o aplicativo e computadores, sem slides.

CamScanner